

Proyecto Final Visión Artificial: Traductor de Braille

Juan Pérez

Ingeniería de Sistemas y Computación
Universidad Tecnológica de Bolívar
Cartagena, Bolívar
Email: perezjuan@utb.edu.co

Jesús Pájaro

Ingeniería Mecatrónica
Universidad Tecnológica de Bolívar
Cartagena, Bolívar
Email: jepajaro@utb.edu.co

Nicolas Molina

Ingeniería de Sistemas y Computación
Universidad Tecnológica de Bolívar
Cartagena, Bolívar
Email: nmolina@utb.edu.co

Gregor de la Rosa

Ingeniería Mecánica
Universidad Tecnológica de Bolívar
Cartagena, Bolívar
Email: delarosag@utb.edu.co

Abstract—Este proyecto presenta la implementación de un sistema de dos etapas para la traducción automatizada de braille mediante técnicas de visión artificial. La primera etapa emplea un modelo de detección de objetos basado en YOLOv11n para localizar caracteres braille en imágenes de escenas naturales. La segunda etapa clasifica cada carácter detectado mediante redes neuronales convolucionales, con dos versiones evaluadas: una basada en una arquitectura PyTorch personalizada y otra que utiliza un clasificador YOLOv11n optimizado. Ambos modelos se entrenaron con conjuntos de datos distintos (reales y sintéticos, respectivamente) y se integraron en un servidor web para la interacción del usuario. Se utilizaron métricas de evaluación como mAP, puntuación F1 y precisión para validar el rendimiento del sistema. Los resultados mostraron alta precisión y recuperación en la detección, y un excelente rendimiento de clasificación tras el ajuste. Sin embargo, las limitaciones de los conjuntos de datos (como la falta de contracciones braille y la compatibilidad con solo 26 letras del inglés) afectaron su aplicabilidad en el mundo real. El diseño modular permite futuras mejoras mediante la mejora de los conjuntos de datos o la sustitución de modelos.

I. MARCO TEÓRICO

El desarrollo de modelos de visión artificial basados en redes neuronales convolucionales (CNN) para tareas de detección de objetos, como el reconocimiento óptico de caracteres Braille, requiere del uso de funciones de pérdida, métricas de evaluación y técnicas de optimización que garanticen un desempeño robusto y generalizable.

A. Funciones de pérdida

En modelos como RetinaNet o YOLOv5, se emplean funciones específicas para optimizar distintas etapas del proceso de detección. La box loss mide el error en la predicción de las coordenadas de las cajas delimitadoras respecto a la ubicación real del objeto, utilizando funciones como IoU (Intersection over Union) o CIoU (Complete IoU). La cls loss corresponde a la pérdida de clasificación, que penaliza errores al asignar la clase correspondiente al objeto detectado, comúnmente basada en entropía cruzada. Por su parte, la dfl loss (Distribution Focal Loss) mejora la precisión en la regresión de coordenadas

utilizando una distribución probabilística sobre los bordes de las cajas, lo que permite una localización más precisa [1], [2].

B. Métricas de evaluación

Para evaluar el rendimiento de un modelo de detección, se utilizan métricas como accuracy, que indica la proporción de predicciones correctas sobre el total, y métricas derivadas de la matriz de confusión como precisión, recall y F1-score. La precisión indica cuántas de las detecciones positivas fueron correctas, mientras que el recall mide cuántos objetos reales fueron efectivamente detectados. El F1-score es la media armónica de ambas, útil en escenarios donde se requiere un balance entre falsos positivos y falsos negativos. En el caso particular de detección de objetos, se utilizan métricas especializadas como mAP@50 (mean Average Precision con umbral IoU del 50%) y mAP@50-95, que promedia el desempeño en diferentes umbrales IoU (de 0.5 a 0.95), proporcionando una evaluación más exigente y detallada [3], [4].

C. Técnicas de mejora del modelo

Durante el entrenamiento, se aplican técnicas como la aumentación de datos (data augmentation), que consiste en modificar las imágenes de entrada mediante transformaciones geométricas, ajustes de brillo o adición de ruido. Esto mejora la capacidad de generalización del modelo al simular distintas condiciones de captura [5]. Por otro lado, el fine-tuning o ajuste fino consiste en reutilizar pesos preentrenados de un modelo base y ajustarlos a un nuevo conjunto de datos mediante un entrenamiento parcial, acompañado del ajuste de hiperparámetros como tasa de aprendizaje, tamaño de lote, número de épocas, entre otros [6]. Estas técnicas permiten optimizar el rendimiento sin necesidad de entrenar desde cero, especialmente útil cuando se dispone de conjuntos de datos limitados, como en el caso del reconocimiento Braille.

II. METODOLOGÍA

La idea desde el inicio del proyecto era la siguiente: subir una imagen con caracteres Braille y luego retornar otra imagen

que contenga los *bounding boxes* de cada carácter, con la traducción respectiva encima de cada uno.

Para el correcto desarrollo del proyecto se barajaron diferentes maneras de llevarlo a cabo, y se determinó que, además del funcionamiento adecuado, se priorizaría que todo el código fuera reutilizable. De esta manera, ante posibles cambios futuros (en el dataset o en los modelos), no sería necesario modificar mucho el código. Esto cobrará relevancia más adelante, pero fue precisamente por este razonamiento que se decidió desarrollar dos modelos: uno encargado de la detección de los caracteres Braille, al que en adelante se hará referencia, de manera indistinta, como modelo uno o primer modelo; y otro encargado de la clasificación de los caracteres, al que se referirá como modelo dos o segundo modelo, también de forma intercambiable.

Además, durante todo el proyecto se priorizó mostrar un resultado funcional, por encima de contar con un buen dataset. Esto no es una buena práctica en realidad, pero debido a que se trata de un proyecto con fines educativos y con limitaciones de tiempo, se tomó esta decisión. Sin embargo, esta elección tendrá sus consecuencias, como se verá más adelante.

A. Extracción de datos

Como se tienen dos modelos diferentes, se deben utilizar también dos datasets distintos para el entrenamiento de cada uno.

Para el primer modelo se utilizó el dataset *Natural Scene Braille Character Recognition Dataset* [7]. Este dataset fue crucial por dos motivos: contenía caracteres Braille en situaciones de la vida real, lo que hace que el modelo sea más robusto; y los caracteres venían etiquetados con su respectivo *bounding box* en formato PascalVOC, y en archivos separados para entrenamiento (925 imágenes) y validación (232 imágenes), lo que facilitó en gran medida su uso.

Para el segundo modelo se utilizó un dataset obtenido directamente desde Kaggle: *Braille Character Dataset* [8]. A diferencia del dataset utilizado para el primer modelo, este está compuesto completamente por imágenes sintéticas, es decir, generadas artificialmente (1560 imágenes en total). Por tanto, se ignoran problemas comunes como la iluminación, lo que puede generar dificultades al traducir imágenes reales. Además, este dataset presenta tres limitaciones importantes que deben tenerse en cuenta: idioma, grados y caracteres.

- 1) Si bien el Braille como tal no es un idioma, los caracteres pueden tener significados completamente distintos dependiendo del alfabeto en que se utilice este sistema. En este caso, el alfabeto utilizado es el **inglés**. Esta limitación no es relevante para el primer dataset, ya que en el primer modelo solo se busca detectar caracteres, no clasificarlos.
- 2) Por otro lado, tanto en Braille como en los idiomas, existen abreviaciones. Así como en inglés se puede abreviar “It will be okay” como “It’ll be okay”, en Braille estas abreviaciones se conocen como grados. Este dataset no toma en cuenta dichos grados.

- 3) Finalmente, el tercer problema es que el dataset solo contiene letras, en concreto: “*abcdefghijklmnopqrstu-vwxyz*”, ignorando números y otros caracteres.

Naturalmente, surge la pregunta de por qué se eligió este dataset, considerando todas las limitaciones mencionadas anteriormente. La elección de este dataset no fue una decisión que nos entusiasmara desde el principio, pero se tomó en función de las prioridades ya mencionadas: mostrar un resultado. Al final, queríamos demostrar lo que se podía lograr con la arquitectura utilizada en el proyecto. Por tanto, la solución a todas estas limitaciones bastaría con reemplazar el dataset del segundo modelo, y todo seguiría funcionando como se espera. Esta fue, en parte, una de las razones por las que se decidió utilizar dos modelos diferentes. Además, si más adelante se desea adaptar el segundo modelo para traducir Braille en otro idioma —por ejemplo, ruso—, este cambio será más sencillo de implementar.

B. Procesamiento de datos

En esta sección se hablará acerca de todo el procesamiento realizado previo al entrenamiento del primer modelo y del segundo modelo.

1) *Detección*: El preprocesamiento de los datos en este modelo fue relativamente sencillo. Únicamente se cambió el tamaño de las imágenes a **640x640** píxeles, agregando cierto *padding* en negro, como se muestra en la figura 1. El reajuste de los tamaños se hizo respetando el etiquetado ya existente. Se ajustaron tanto las dimensiones de cada imagen como sus etiquetas, y estas últimas se exportaron del formato PascalVOC al formato YOLO.

2) *Clasificación*: Hasta este punto no se ha mencionado el versionado que se aplicó a este segundo modelo; las razones se explicarán en la sección II-C. En este caso, el preprocesamiento fue más extenso que en el dataset del modelo anterior, y consistió en los siguientes pasos:

- 1) **Filtros**. El dataset de Kaggle consistía en imágenes sintéticas de 28x28 píxeles en escala de grises, con aumentaciones incluidas como desplazamientos en anchura y altura, rotación y brillo. Inicialmente se aplicaron filtros de limpieza (filtro de la mediana) y de realce de bordes al dataset original.
- 2) **Aumentación de datos**. Tras aplicar los primeros filtros, el resultado no fue del todo satisfactorio, por lo que se decidió ampliar el conjunto de datos con nuevas transformaciones. A cada imagen del dataset se le aplicaron diferentes modificaciones: cambio de color de los caracteres (puntos blancos o grises con fondo oscuro, entre otros), adición de ruido y desenfoque, aumento del contraste, simulación de brillo y transformaciones de perspectiva (nunca se aplicó rotación). Luego, a cada una de las imágenes resultantes se les invirtieron los colores. El resultado final fue un conjunto de 28,080 imágenes, es decir, 1,080 por cada letra. De estas, 21,060 (75%) se utilizaron para entrenamiento, y 7,020 (25%) para validación.



(a) Extraída directamente desde el dataset.
Tamaño real: 77x60 píxeles



(b) Preprocesada. Tamaño real: 640x640 píxeles

Fig. 1: Imagen 5_0.jpg antes y después del preprocesamiento.

- 3) **Cambio de tamaño.** Dependiendo de la versión del modelo, se ajustó el tamaño de las imágenes. Para la versión 1, se mantuvo el tamaño original: **28x28**. Para la versión 2, las imágenes fueron redimensionadas a **32x32**.



(a) Imagen original.



(b) Imagen preprocesada con ajuste de perspectiva.



(c) Imagen preprocesada con ajuste de perspectiva y colores invertidos.

Fig. 2: Letra “a” extraída directamente del dataset de Kaggle y ejemplos de aumentación de datos.

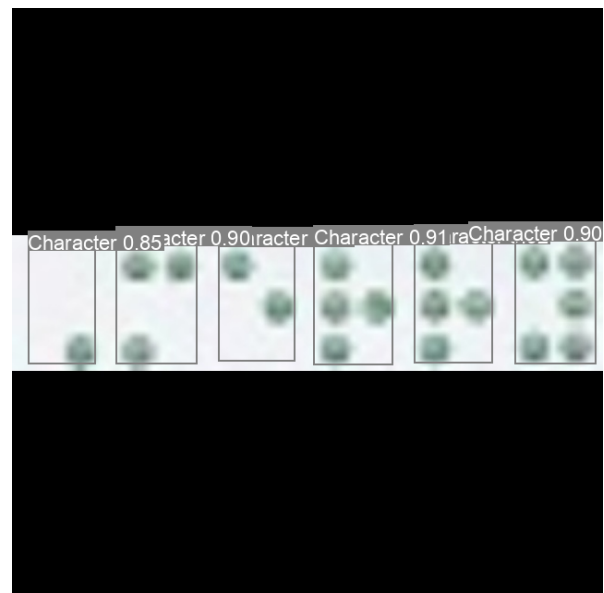
C. Modelos

Esta sección está completamente relacionada con el entrenamiento del primer y segundo modelo.

1) **Detección:** En primer lugar, se descargó un modelo preentrenado de YOLOv11n para detección de objetos, y se reentrenó con nuestro dataset. En este entrenamiento no se aplicaron técnicas de *fine-tuning*, únicamente se especificó que se estaría detectando una única clase: la clase carácter. En la figura 3 se muestra un ejemplo de su funcionamiento, indicando la clase detectada y el grado de confianza.



(a) Imagen original.



(b) Caracteres detectados.

Fig. 3: El primer modelo detectando los caracteres de la imagen original, mostrando la clase “Character” y la confianza.

2) Clasificación: Versión 1

En el apartado de códigos hechos por la comunidad en el dataset obtenido de Kaggle [8], se tomó una de las soluciones como inspiración para realizar el modelo de la versión 1 [9].

Este modelo estaba escrito en Python 3 utilizando TensorFlow, pero, debido a las ventajas del uso de GPU en contextos de *machine learning*, se decidió migrar el código a PyTorch, manteniendo esencialmente la misma lógica. De este modelo se obtuvieron resultados bastante buenos, los cuales se analizan en la sección III-B.

Versión 2

¿Por qué una segunda versión? La razón principal es que el grado de confianza obtenido en esta versión resultó más “seguro” que el alcanzado en la versión 1. Así que, con el objetivo de explorar distintas alternativas, quisimos replicar algo similar al enfoque usado en el primer modelo (detección), pero aplicado ahora a la clasificación. Para ello, se descargó un modelo preentrenado de YOLOv11n para clasificación y se entrenó con el mismo dataset utilizado en la versión 1, con sus respectivos cambios en la estructura del conjunto de datos. En esta ocasión, se trabajó con imágenes de tamaño **32x32** píxeles.

¿Por qué 32x32? Porque YOLO, en sus modelos, requiere que el tamaño de entrada sea múltiplo de 32 (esto está codificado directamente en su estructura), por lo que fue más práctico reajustar las imágenes a 32x32 en lugar de mantenerlas en 28x28. Los resultados de esta versión pueden encontrarse en la sección III-B.

En esta versión sí se aplicó *fine-tuning*, que consistió en ajustar la tasa de aprendizaje inicial a 0.001, fijar el optimizador a *Adam*, y aplicar aún más aumentación de datos. Para entrenar modelos preentrenados en YOLO, este framework proporciona una serie de hiperparámetros destinados a mejorar la robustez mediante aumentaciones. En este caso, se configuraron las variaciones de *hue* y *saturation* a 0.1 y 0.8 respectivamente [10]. Además, se utilizó el método de aumentación *AugMix* [11] durante el entrenamiento de esta versión. Finalmente, se aplicó un *dropout* de 0.2.

D. Integración

Finalmente, se integraron los modelos resultantes en el archivo `main.py`, teniendo en cuenta detalles como las versiones de los modelos de clasificación. Además, relacionado con lo mencionado al inicio de esta sección, se procuró —y se consiguió— que el código fuera escalable, haciendo uso de interfaces y otros conceptos de Arquitectura de Software.

Con el fin de poder probar la aplicación, se creó un pequeño servidor web en Python utilizando FastAPI, con una página estática donde se puede subir una imagen con caracteres en Braille y obtener como resultado su traducción.

Todo el código fuente está alojado en GitHub y se puede acceder desde el siguiente enlace: <https://github.com/dosquisd/vision-artificial-utb/tree/main/project> [12]. Además, se utilizó Docker para facilitar el despliegue del servidor web, creando una imagen que está publicada en DockerHub, accesible desde el siguiente enlace: <https://hub.docker.com/r/dosquisd/braille-translator> (ver figura 4).

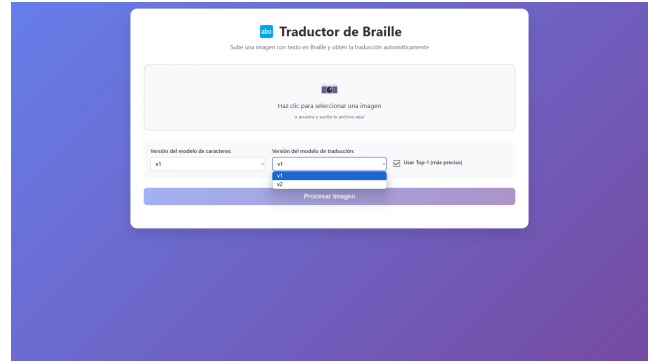


Fig. 4: Servidor web para subir imágenes en Braille y obtener su traducción.

III. RESULTADOS

En esta sección se presentan los resultados obtenidos por cada uno de los modelos, y se analizan sus métricas respectivas.

A. Detección

Durante el entrenamiento del modelo de detección, se evaluaron los resultados a partir de las gráficas generadas durante dicho proceso. Las figuras 5 y 6 son algunas de las más representativas.

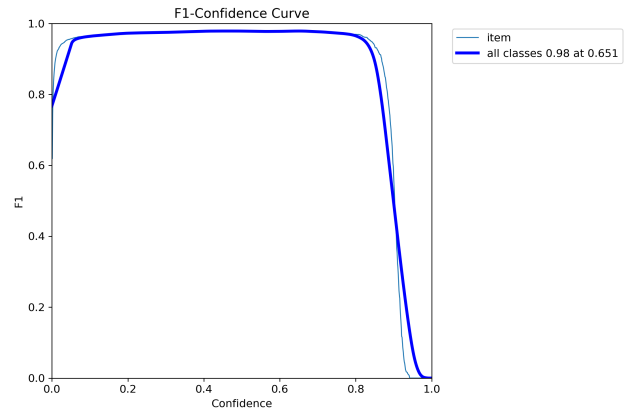


Fig. 5: *F1* vs. *Confidence*

Respecto a la figura 5, se observa la evolución de la métrica *F1* en función de la confianza. Para valores bajos de confianza, el *F1* crece desde un poco menos de 0.8 hasta casi 1; luego se mantiene constante cerca de 1 y finalmente cae cuando la confianza es muy alta (mayor a 0.9, aproximadamente). Esta caída se debe a que, al exigir una confianza muy alta, la precisión tiende a 1, pero el *recall* cae drásticamente, lo que provoca una disminución en la métrica *F1*.

En la figura 6, se muestran las funciones de pérdida durante el entrenamiento y la validación a lo largo de todas las épocas. El comportamiento observado es el esperado: a medida que avanzan las épocas, el error tiende a disminuir. Sin embargo, vale la pena analizar cada una de las pérdidas por separado:

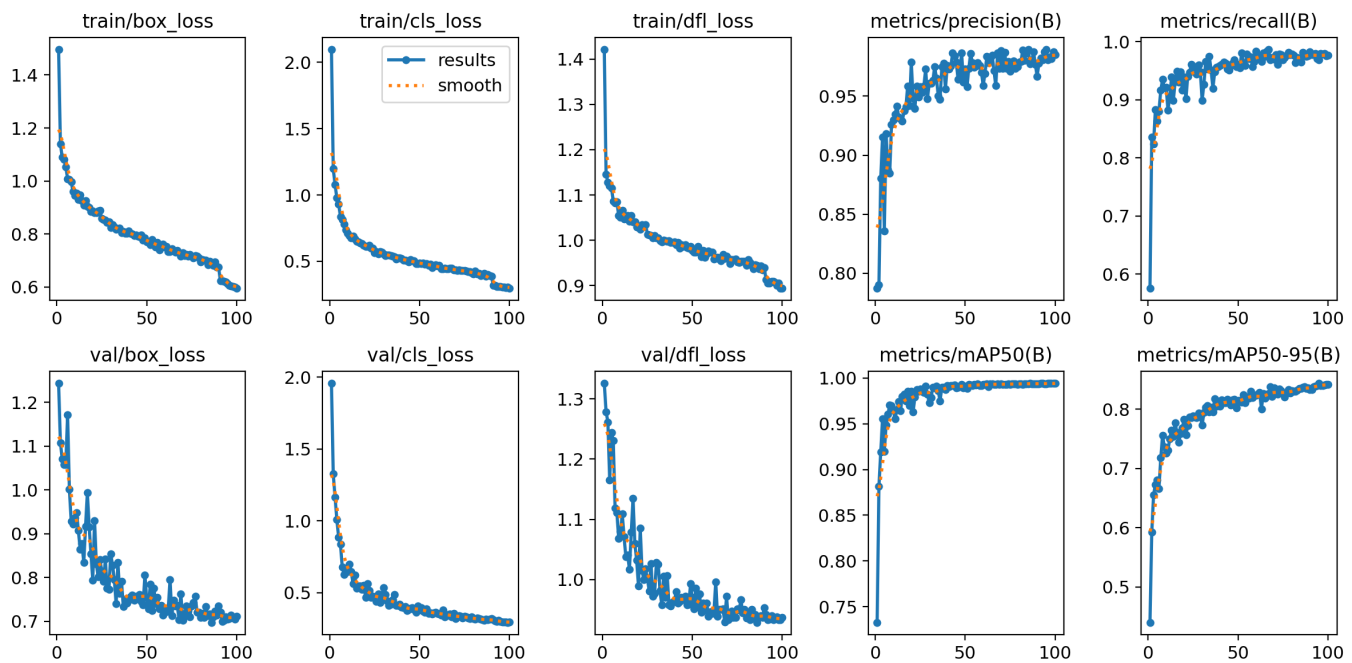


Fig. 6: Gráficas resultantes del entrenamiento del primer modelo.

1. **box_loss**. Relacionada con el error al predecir las coordenadas de los *bounding boxes*. Este valor resulta algo alto, tendiendo a 0.6 en entrenamiento y 0.7 en validación, lo que sugiere que el modelo aún tiene dificultades para ubicar correctamente los caracteres.

2. **cls_loss**. Mide el error de clasificación del objeto, a pesar de que solo se trabaja con una única clase (carácter), ya que también se debe distinguir entre fondo y objeto. Durante el entrenamiento y validación, esta pérdida tiende a 0.4, lo cual se considera un valor razonablemente bueno.

3. **df_l_loss**. Evalúa qué tan bien el modelo aprende a predecir las distancias de los bordes de las cajas al centro del objeto. Esta pérdida alcanza valores cercanos a 0.9 tanto en entrenamiento como en validación, lo que indica que aún hay dificultades para delimitar con precisión los *bounding boxes*.

Por último, en las dos últimas columnas del gráfico se reportan las métricas de precisión, *recall*, mAP@50 y mAP@50-95. Estas últimas dos se refieren a la *mean Average Precision*, utilizando umbrales de IoU de 0.5 (para mAP@50) y desde 0.5 hasta 0.95 con pasos de 0.05 (para mAP@50-95).

1. **Precisión**. Mide la proporción de predicciones positivas que son correctas. Valores altos indican menos falsos positivos. En este caso, la precisión tiende a 1.

2. **Recall**. Mide la proporción de verdaderos positivos detectados. Valores altos indican menos falsos negativos. En este modelo, el *recall* también tiende a 1.

3. **mAP**. La media de precisión promedio por clase, evaluada con distintos umbrales de IoU. Se obtuvieron resultados excelentes, con mAP@50 tendiendo a 1, y mAP@50-95 alcanzando aproximadamente 0.9.

B. Clasificación

Versión 1

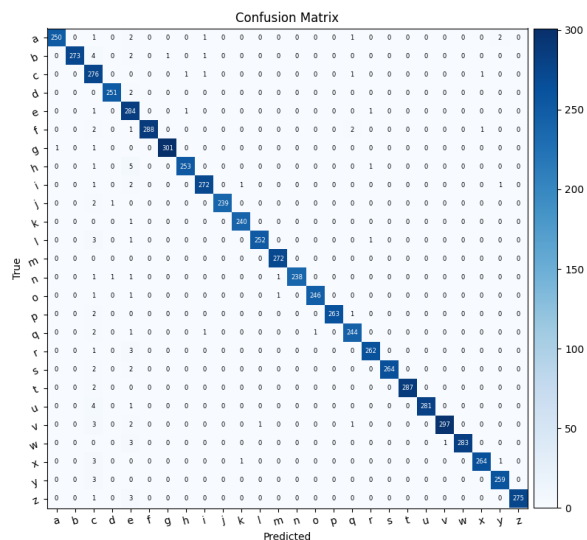
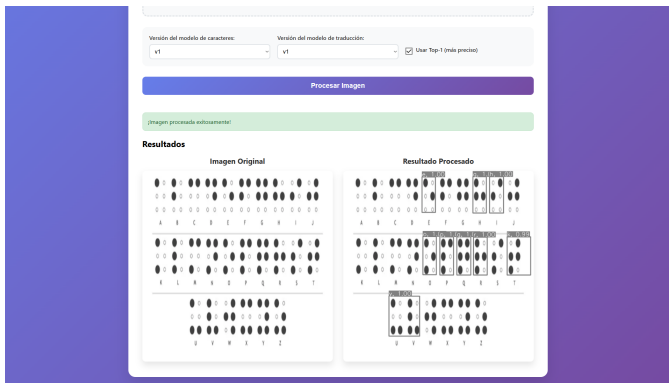
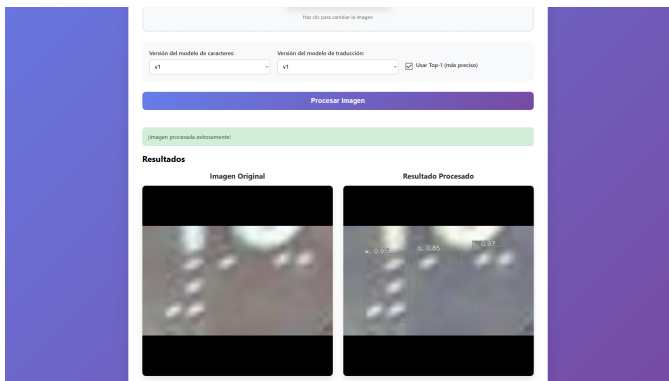


Fig. 7: Matriz de confusión en la primera versión del modelo de clasificación.

Los resultados obtenidos con este modelo fueron bastante



(a) Errores en el primer modelo. Umbral de confianza e IoU en 0.8 y 0.7 respectivamente.



(b) Errores en el segundo modelo. Umbral de confianza e IoU en 0.5 y 0.7 respectivamente en el primer modelo.

Fig. 9: Servidor web donde se sube una imagen para validar su funcionamiento.

box_loss y dfl_loss sugieren que el modelo presenta dificultades para ubicar con precisión los bounding boxes, lo que podría mejorarse con entrenamientos más extensos o ajustes en los hiperparámetros.

Para el modelo de clasificación se desarrollaron dos versiones. La primera versión, inspirada en una solución de la comunidad de Kaggle y migrada a PyTorch, alcanzó una accuracy de 0.9855 y un F1-score de 0.9858. La segunda versión, también basada en YOLOv11n, mostró la importancia del fine-tuning, mejorando de una accuracy de 0.9305 a 0.9762 tras aplicar técnicas de ajuste fino.

La decisión de utilizar dos modelos separados se justificó por la flexibilidad que esto proporciona ante cambios futuros en datasets o requerimientos del sistema. Esta arquitectura modular permite, por ejemplo, reemplazar únicamente el modelo de clasificación para adaptar el sistema a otros idiomas sin modificar el modelo de detección.

Durante el desarrollo del proyecto se priorizó mostrar un resultado funcional por encima de contar con datasets de alta calidad. Esta elección tuvo consecuencias importantes: el dataset utilizado para clasificación, compuesto por imágenes

sintéticas de solo 26 letras del alfabeto inglés, sin considerar números, grados de Braille ni otros caracteres, limitó significativamente la aplicabilidad del sistema en escenarios reales.

Las pruebas de integración en el servidor web confirmaron las limitaciones identificadas en ambos modelos. El primer modelo mostró dificultades para delimitar correctamente los bounding boxes, como se había anticipado por los valores altos de dfl_loss, resultando en detecciones imprecisas o pérdida de caracteres cuando el umbral de confianza se establece en valores altos como 0.8. Por su parte, el segundo modelo exhibió errores de clasificación incluso cuando la detección fue exitosa, clasificando incorrectamente caracteres como "c" predicho como "b", además de enfrentarse a caracteres que no existen en el alfabeto inglés utilizado para su entrenamiento.

Estas limitaciones se manifestaron de manera práctica al probar el sistema completo: tanto errores individuales de cada modelo como fallos combinados que comprometen la traducción final. La diferencia entre las imágenes sintéticas utilizadas para entrenamientos y las imágenes reales, junto con las restricciones del dataset de clasificación, resultan en un rendimiento deficiente que limita la utilidad práctica de la herramienta en aplicaciones del mundo real.

El proyecto demostró que es factible implementar un sistema de traducción Braille automatizado utilizando modelos de detección y clasificación por separado, y que la arquitectura modular facilita futuras mejoras. Sin embargo, también evidenció que la calidad de los datos es fundamental para el éxito de cualquier sistema de machine learning, y que priorizar resultados funcionales por encima de datasets robustos puede limitar severamente la aplicabilidad real del sistema desarrollado.

REFERENCES

- [1] G. Jocher, A. Chaurasia, T. Qiu, *et al.*, "Yolov5 by ultralytics." <https://github.com/ultralytics/yolov5>, 2023. GitHub repository.
- [2] T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, "Focal loss for dense object detection," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 42, no. 2, pp. 298–307, 2020.
- [3] Papers with Code, "Mean average precision (map)." <https://paperswithcode.com/method/mean-average-precision>, 2023.
- [4] C. Consortium, "Coco dataset - evaluation metrics." <https://cocodataset.org/#detection-eval>, 2023.
- [5] C. Shorten and T. M. Khoshgoftaar, "A survey on image data augmentation for deep learning," *Journal of Big Data*, vol. 6, no. 1, p. 60, 2019.
- [6] T. Developers, "Transfer learning and fine-tuning." https://www.tensorflow.org/tutorials/images/transfer_learning, 2023.
- [7] lu li qiong, W. Dong, and X. Jianfang, "Natural scene braille character recognition dataset." <https://doi.org/10.57760/sciencedbj00188.00007>.
- [8] Shanks0465, "Braille character dataset." <https://www.kaggle.com/datasets/shanks0465/braille-character-dataset>.
- [9] O. S. Parulian, "Efficient design and compression of cnn models for rapid character recognition au," *Jurnal Ilmu Komputer dan Informatika*, 2025.
- [10] U. team, "Model training with ultralytics yolo." <https://docs.ultralytics.com/modes/train/#train-settings>.
- [11] D. Hendrycks, N. Mu, E. D. Cubuk, B. Zoph, J. Gilmer, and B. Lakshminarayanan, "Augmix: A simple data processing method to improve robustness and uncertainty," 2020.
- [12] J. Perez, J. Pajaro, N. Molina, and G. de la Rosa, "Traductor de braille." <https://github.com/dosquid/vision-artificial-utb/tree/main/project>.